

StudyMate-RAG v2.0

End-to-End AIML Architecture & Deployment Guide

Master every single line of code, design pattern & recruiter question.

Prepared by: Jashwanth Kondasani

• 1. What is StudyMate-RAG in Plain Words?

StudyMate-RAG is an AI-powered personal study tutor that ingests PDF documents (lecture notes, textbook chapters, syllabi) and answers student questions in plain English. Unlike generic AI chatbots that answer from general memory (prone to hallucinations and outdated facts), StudyMate-RAG grounds every single answer STRICTLY inside your uploaded document pages and provides page-level citations.

• Think Of It Like This:

It is like hiring a personal tutor who has read your exact textbook page by page. During an open-book exam, instead of trying to answer from memory, the tutor turns directly to the exact pages containing the answer, highlights the facts, and says: 'Here is your answer, based strictly on Page 12 of your notes.' If the textbook doesn't cover something, the tutor honestly states 'That is not mentioned in your notes' rather than making things up.

• Key Highlights of System Architecture

- PyMuPDF Text Extraction: Fast page-by-page text parsing with exact page metadata.
- High-Precision Chunking: 800-character sliding windows with 150-character overlap.
- Dual Vector Embeddings: Google Gemini (text-embedding-004) + Local Neural SentenceTransformers.
- ChromaDB Vector Database: Fast cosine-similarity vector index and metadata retrieval.
- Anti-Hallucination Grounding Prompt: Prevents model from fabricating outside facts.
- Advanced Feature Suite: Multi-PDF search, Multi-turn memory, Voice TTS, AI Quizzes & Flashcards.
- Full-Stack Deployment: Mobile-responsive Streamlit UI + FastAPI REST API + Docker setup.

... Developer & Author Profile

Name: Jashwanth Kondasani

Email: jashwanthkumarreddy53@gmail.com

GitHub: <https://github.com/jashwanthkondasani>

LinkedIn: <https://www.linkedin.com/in/jashwanth-reddy-3919152a0>

2. Full Tech Stack · Every Tool Explained

Here is every single piece of technology in your project and why it was chosen over alternatives:

Piece	Its Job In Plain Words	Why This Choice Over Alternatives
Python 3.11+	Core programming language gluing components together.	Standard language for AI/ML with rich library ecosystem.
PyMuPDF (fitz)	Opens PDF files and extracts text page-by-page.	Blazing fast, handles scanned/complex PDFs with page metadata.
Text Chunker	Splits long text into overlapping 800-char pieces.	Keeps sentences contiguous so no idea gets awkwardly chopped.
SentenceTransformers	Generates 384-dim neural semantic vectors locally.	Provides deep neural vector retrieval without requiring API key.
Gemini Embeddings	Generates 768-dim semantic vectors via Google API.	Captures conceptual meaning across hundreds of dimensions.
ChromaDB	Vector database storing embeddings & metadata.	Purpose-built for fast cosine similarity search at scale.
Gemini 2.5 Flash	Reads question + retrieved chunks to write answer.	Free tier available, strong quality, strict grounded prompt following.
FastAPI	Backend REST API server receiving requests.	Modern, fast async framework with automatic Swagger docs.
Streamlit v2.0	Responsive web and mobile frontend UI.	Lets you build a sleek UI in Python with responsive CSS.
Docker	Packages app code & dependencies into portable box.	Guarantees identical execution on laptop and cloud servers.

3. The 10-Step Journey of One Question

If a recruiter or guide asks: 'Walk me through what happens when a user asks a question', recite this:

STEP 1 • PDF Upload

The user uploads a PDF (notes/textbook/syllabus) via Streamlit UI or FastAPI endpoint.

STEP 2 • PyMuPDF Text Extraction

PyMuPDF opens the PDF and extracts raw text page-by-page, attaching page number metadata.

STEP 3 • Text Chunking

Text is cut into overlapping 800-character chunks with 150-char overlap to preserve context.

STEP 4 • Vector Embedding Generation

Each text chunk is converted into a numeric embedding vector representing its semantic meaning.

STEP 5 • ChromaDB Vector Storage

Text chunks, embedding vectors, and page metadata are stored and indexed in ChromaDB.

STEP 6 • User Question

The user types a question in the chat box (e.g. 'What is photosynthesis?').

STEP 7 • Question Embedding

The exact same vector model converts the question into a numeric query vector.

STEP 8 • Similarity Search

ChromaDB performs cosine similarity search to find top-5 chunks closest in meaning to the query.

STEP 9 • Grounded Prompt Construction

Retrieved chunks are combined into a strict prompt: 'Answer using ONLY these snippets. Say I don't know if missing.'

STEP 10 • Gemini LLM Generation

Gemini LLM generates a natural language answer with page citations [Page X], rendered in the UI.

4. The 6 Concepts You Must Explain Confidently

4.1 Embeddings

A numeric representation of meaning. Two pieces of text with similar meaning get numerically similar embeddings, even if they don't share exact words.

Q. What is an embedding in your own words?

A. It's a way of turning text into numbers that capture meaning so computers can measure concept similarity.

4.2 Vector Database (ChromaDB)

A database specifically built to store numeric embeddings and quickly answer 'which stored items are most similar to this new vector?'

Q. Why not use a regular SQL database here?

A. SQL is built for exact text matches, whereas vector DBs compare high-dimensional meaning spaces efficiently.

4.3 Text Chunking

Splitting a long document into smaller overlapping pieces before embedding and storing them.

Q. Why split into chunks instead of embedding the whole PDF?

A. A single embedding for an entire document blurs topics together. Chunks allow precise paragraph retrieval.

4.4 Similarity Search

Comparing the question's embedding against stored chunk embeddings using Cosine Distance.

Q. How does ChromaDB decide which chunks are relevant?

A. It measures the cosine angle between query vector and stored vectors, returning the closest matches.

4.5 Context Window

The maximum amount of text an LLM can read at once in a single prompt.

Q. Why use RAG instead of feeding the whole PDF to LLM?

A. Large documents exceed context limits, and focused relevant text produces more accurate answers.

4.6 Hallucination & Grounding

Hallucination is when an LLM confidently invents false facts. Grounding forces answers to use retrieved snippets.

Q. How does your project reduce hallucination?

A. By instructing Gemini to answer ONLY from retrieved chunks and say 'I don't know' if missing.

5. Recruiter & Interviewer Q&A Bank

Q. Walk me through your project in 90 seconds.

A. StudyMate-RAG is a grounded study assistant where students upload PDF notes and ask questions in plain English. Under the hood, it parses text using PyMuPDF, chunks it into overlapping 800-character windows, generates semantic embeddings using Google Gemini and SentenceTransformers, and indexes them in ChromaDB. When a user asks a question, ChromaDB retrieves top relevant chunks via cosine similarity search, and Google Gemini synthesizes an accurate answer grounded strictly in the document with page citations.

Q. Why RAG instead of Fine-Tuning?

A. Fine-tuning is expensive, slow to retrain whenever notes change, and still hallucinates. RAG allows instant document swapping with zero retraining costs and provides exact page-level source traceability.

Q. What was the hardest part of building this?

A. Tuning chunk size and overlap so retrieval was precise without losing sentence context across chunk boundaries, and implementing multi-turn memory without exceeding prompt bounds.

Q. What happens if the PDF doesn't contain the answer?

A. The grounded prompt explicitly instructs Gemini to reply 'I cannot find the answer to this question in your uploaded document' rather than guessing from general knowledge.

Q. How did you deploy this project?

A. Pushed code to GitHub, managed dependencies in requirements.txt, secured API keys in .env / Streamlit secrets, and deployed on Streamlit Community Cloud and Docker.

6. Detailed Step-by-Step Deployment Guide

Here are the 3 complete ways to deploy StudyMate-RAG v2.0 for mobile and public access:

OPTION 1 · Streamlit Community Cloud (Free Public Link)

1. Push code to GitHub: `git init` && `git add .` && `git commit -m 'Deploy'` && `git push origin main`
2. Connect to share.streamlit.io, select repository and set main file path to `app.py`.
3. Add GEMINI_API_KEY under Advanced Settings -> Secrets and click Deploy.
4. Get a live URL (e.g. <https://studymate-rag.streamlit.app>) accessible on any mobile phone browser!

OPTION 2 · Docker Container Production Deployment

1. Build Docker image: `docker build -t studymate-rag:v2 .`
2. Run container: `docker run -d -p 8501:8501 -e GEMINI_API_KEY='your_key' studymate-rag:v2`
3. Deploy using Docker Compose: `docker-compose up --build -d`
4. Host on Render, Railway, or AWS EC2 for production container execution.

OPTION 3 · Instant Mobile Access via Local Tunneling

1. Ensure local Streamlit server is running: `streamlit run app.py --server.port 8501`
2. Run local tunnel command in terminal: `npx localtunnel --port 8501`
3. Open generated public URL on your phone's Safari/Chrome browser to upload PDFs and ask questions on the go!